Product Requirements Document

Upper Peninsula Trip Planner

TypeScript React Application

8-Day Road Trip — Grosse Pointe Park to Porcupine Mountains and Back

Version 1.0 | March 2026

Status: Draft

1. Overview

This document defines the requirements for a TypeScript React application that serves as an interactive trip planner for a road trip through Michigan's Upper Peninsula. The app will display a map of Michigan showing the driving route with markers for each overnight stop, and present the full itinerary in an editable, data-driven format.

Primary Goal: Build a single-page React application that visualizes an 8-day UP road trip on an interactive map, with a structured itinerary view driven by easily-modifiable TypeScript data structures.

Target Users: Trip participants who need a clear visual overview of the route, schedule, and logistics.

Tech Stack: TypeScript, React 18+, Leaflet.js (or Mapbox GL JS), Tailwind CSS

2. Trip Itinerary

The following table represents the complete schedule. Each row maps to a TripDay object in the application's data layer. The schedule spans Day 5 (Saturday) through Day 12 (Saturday), totaling 8 travel days and approximately 1,575 miles.

Day	DOW	From	To	Hrs	Miles	Notes / Overnight
5	Sat	HOME (Grosse Pointe Park)	Tahquamenon Falls	5.5	360	Depart early / Explore falls / Camp at T.Falls St. Park — T.Falls State Park Campground
6	Sun	Tahquamenon Falls	Pictured Rocks	1.5	90	Leave falls anytime / Camp near Pictured Rocks — Pictured Rocks Campground
7	Mon	Pictured Rocks	Pictured Rocks	0	0	Full day at Pictured Rocks / Excursions (kayak/boat tour) — Pictured Rocks Campground
8	Tue	Pictured Rocks	Houghton	2.5	150	Stop in Marquette (mining tour or other) / Hotel in Houghton — Houghton Hotel/BnB
9	Wed	Houghton	Copper Harbor	1	50	Leave Houghton early / Spend day in Copper Harbor — Copper Harbor Campground
10	Thu	Copper Harbor	Porcupine Mountains	2.5	125	Arrive at Porkies mid-day / Setup camp / Relax + sunset — Porkies Campground
11	Fri	Porcupine Mountains	Porcupine Mountains	0	0	Full day at Porkies / Hike trails or peak — Porkies Campground
12	Sat	Porcupine	HOME	10	600	Full drive back w/ breaks (N. or S. route)

		Mountains				— Home / Optional stay near Mackinaw

3. Core Data Structures

The application's state is driven by a small set of TypeScript interfaces. These are designed so that schedule changes require editing only one data file, with the map and itinerary views reacting automatically.

3.1 Location

Represents any named point on the trip. Coordinates are required for map placement.

```
interface Location {
  id: string;                    // unique slug, e.g. 'tahquamenon-falls'
  name: string;                  // display name
  shortName: string;             // abbreviated label for map markers
  lat: number;                   // latitude
  lng: number;                   // longitude
  type: 'home' | 'campground' | 'hotel' | 'attraction' | 'city';
  description?: string;          // optional blurb
  website?: string;              // optional link
  phone?: string;                // reservation contact
  reservationConfirmed?: boolean; // booking status flag
}
```

3.2 TripDay

Each day of the trip. Drives, stops, and overnight info are all captured here. When the schedule changes, update the TRIP_DAYS array and everything flows through.

```
interface TripDay {
  dayNumber: number;             // calendar day (5–12)
  dayOfWeek: DayOfWeek;          // 'Sat' | 'Sun' | ... | 'Fri'
  date: string;                  // ISO date, e.g. '2026-07-05'
  departure: Location;           // where we leave from
  destination: Location;         // where we end up
  drivingHours: number;          // estimated drive time
  drivingMiles: number;          // estimated mileage
  overnight: OvernightStop;      // where we sleep
  activities: Activity[];        // planned things to do
  notes: string;                 // free-text notes
  routeWaypoints?: [number, number][]; // optional intermediate coords
}
```

3.3 OvernightStop

Describes where the group sleeps each night. The type field drives the marker icon on the map.

```
interface OvernightStop {
  location: Location;
  type: 'campground' | 'hotel' | 'bnb' | 'home';
  reservationId?: string;
  checkIn?: string;              // e.g. '3:00 PM'
  checkOut?: string;
  cost?: number;                 // per night
  notes?: string;
}
```

3.4 Activity

Individual things to do at a stop. Optional fields allow cost and time tracking.

```
interface Activity {
  id: string;
  name: string;                  // e.g. 'Kayak Tour at Pictured Rocks'
  location: Location;
  category: 'hike' | 'tour' | 'water' | 'food' | 'scenic' | 'other';
  estimatedDuration?: number;    // minutes
  cost?: number;
  booked?: boolean;
  url?: string;
  notes?: string;
}
```

3.5 TripConfig (Top-Level)

The root object the application consumes. A single export from a data file.

```
interface TripConfig {
  tripName: string;
  startDate: string;             // ISO date of Day 1
  endDate: string;
  participants: string[];
  days: TripDay[];
  locations: Location[];          // master location registry
  totalMiles: number;             // computed
  totalDrivingHours: number;      // computed
}
```

4. Location Registry

Below is the seed data for all locations referenced in the trip. These coordinates will be used to place markers on the map and to compute driving segments between stops.

Location	Lat	Lng	Type	Short	Notes
Grosse Pointe Park	42.3756	-82.9346	home	HOME	Start / End
Tahquamenon Falls SP	46.5958	-85.1518	campground	T.FALLS	Upper & Lower Falls
Pictured Rocks NL	46.5575	-86.3500	campground	P.ROCKS	Kayak + boat tours
Marquette	46.5436	-87.3954	city	MQT	Mining tour stop
Houghton	47.1211	-88.5694	hotel	HOU	Hotel or BnB
Copper Harbor	47.4683	-87.8903	campground	C.HARBOR	End of Keweenaw
Porcupine Mountains SP	46.7558	-89.7960	campground	PORKIES	Trails + Lake of the Clouds

5. Feature Requirements

5.1 Interactive Map View

The centerpiece of the application. An interactive map of Michigan (with emphasis on the Upper Peninsula) showing the full driving route and overnight markers.

1. Display a full-screen or near-full-screen interactive map using Leaflet.js with OpenStreetMap tiles (free, no API key required) or Mapbox GL JS.
2. Render the driving route as a colored polyline connecting each stop in order: HOME → Tahquamenon Falls → Pictured Rocks → Houghton → Copper Harbor → Porcupine Mountains → HOME.
3. Place custom markers at each overnight location. Marker icons should differ by OvernightStop.type (tent for campground, bed for hotel/bnb, house for home).
4. Clicking a marker opens a popup showing: location name, day(s) staying, activities planned, overnight type, and any notes.
5. The map should auto-fit bounds to show the entire route on load, with padding so no markers are clipped.
6. Route segments between stops should be drawn as straight polylines using the coordinate pairs from the Location registry. Optionally, integrate a routing API (OSRM or Mapbox Directions) for actual road-following paths.
7. Include a day-by-day highlight mode: user can select a day from the itinerary panel and the map zooms/highlights that segment.

5.2 Itinerary Panel

A sidebar or collapsible panel that presents the trip schedule in a clear, scannable format.

1. Display each TripDay as a card with: day number, date, from/to labels, driving time and distance, overnight info, and activity list.
2. Cards should be visually distinct for travel days vs. stay days (0 miles driven).
3. Show a trip summary header with total mileage, total driving hours, and trip duration.
4. Clicking a day card should highlight the corresponding route segment on the map and open the relevant marker popup.

5.3 Schedule Modification Support

The data architecture must make it trivial to update the schedule without touching component code.

1. All trip data lives in a single file: src/data/tripData.ts. This file exports a TripConfig object.
2. Adding a day: push a new TripDay to the days array. The map and itinerary update automatically.
3. Changing a stop: update the departure/destination Location references and the overnight info.
4. Reordering: rearrange the days array entries. dayNumber is display-only and can be recalculated.
5. Adding a location: add it to the locations registry, then reference it in the relevant TripDay.

5.4 Responsive Layout

1. Desktop (1024px+): side-by-side layout with map on the left (65% width) and itinerary panel on the right (35%).
2. Tablet (768–1023px): map on top, itinerary below, both scrollable.
3. Mobile (<768px): full-width map with a slide-up itinerary drawer.

6. Example Trip Data File

The following shows the structure of src/data/tripData.ts. This is the single source of truth that developers edit when the schedule changes. The full file will contain all 8 days; two representative entries are shown here.

```
// src/data/tripData.ts
import { TripConfig, Location } from '../types';

// ── Location Registry ──
export const LOCATIONS: Record<string, Location> = {
  home: {
    id: 'home',
    name: 'Grosse Pointe Park',
    shortName: 'HOME',
    lat: 42.3756,
    lng: -82.9346,
    type: 'home',
  },
  tahquamenonFalls: {
    id: 'tahquamenon-falls',
    name: 'Tahquamenon Falls State Park',
    shortName: 'T.FALLS',
    lat: 46.5958,
    lng: -85.1518,
    type: 'campground',
    website: 'https://www2.dnr.state.mi.us/parksandtrails/Details.aspx?id=428',
  },
  // ... remaining locations ...
};

// ── Trip Configuration ──
export const TRIP: TripConfig = {
  tripName: 'Upper Peninsula Road Trip 2026',
  startDate: '2026-07-04',
  endDate: '2026-07-12',
  participants: ['...'],
  locations: Object.values(LOCATIONS),
  totalMiles: 1575,
  totalDrivingHours: 23.5,
  days: [
    {
      dayNumber: 5,
      dayOfWeek: 'Sat',
      date: '2026-07-05',
      departure: LOCATIONS.home,
      destination: LOCATIONS.tahquamenonFalls,
      drivingHours: 5.5,
      drivingMiles: 360,
      overnight: {
        location: LOCATIONS.tahquamenonFalls,
        type: 'campground',
        notes: 'State park campground — reserve early',
      },
      activities: [
        {
          id: 'tf-upper',
```

```
          name: 'Upper Falls Viewing',
          location: LOCATIONS.tahquamenonFalls,
          category: 'scenic',
          estimatedDuration: 90,
        },
      ],
      notes: 'Depart early. Explore falls upon arrival.',
    },
    // ... remaining days ...
  ],
};
```

7. Map Implementation Notes

7.1 Recommended Stack

- Mapping library: react-leaflet (wraps Leaflet.js) — free, no API key, robust ecosystem.
- Tile provider: OpenStreetMap (default) or CartoDB Voyager for a cleaner aesthetic.
- Route rendering: connect Location coordinates with L.Polyline. For road-accurate paths, optionally call the free OSRM API.
- Marker icons: use Leaflet.DivIcon with inline SVG or emoji for campground (⛺), hotel (🏨), and home (🏠) types.

7.2 Map Behavior

- On initial load, fit bounds to show the full route with 40px padding.
- When a day is selected in the itinerary, animate the map to show only that day's from/to segment with a 2-second fly-to transition.
- Marker popups should include the day card info and a link that scrolls the itinerary panel to the matching day.
- The route polyline should use a distinct color per segment (or a single color with animated dashes to indicate direction).

8. Component Architecture

The application uses a simple component hierarchy. State is managed with React Context to avoid prop drilling between the map and itinerary.

```
src/
  types/
    index.ts              // All TypeScript interfaces
  data/
    tripData.ts           // TripConfig export (SINGLE SOURCE OF TRUTH)
```

```
context/
  TripContext.tsx        // React Context provider + useTrip hook
components/
  App.tsx               // Root layout
  MapView/
    MapView.tsx         // Leaflet map container
    RoutePolyline.tsx   // Route line rendering
    StopMarker.tsx      // Individual marker + popup
  Itinerary/
    ItineraryPanel.tsx  // Sidebar container
    DayCard.tsx         // Single day summary card
    TripSummary.tsx     // Total stats header
  shared/
    Icons.tsx           // Marker icon components
```

9. Non-Functional Requirements

- Performance: Map tiles should lazy-load. Initial render under 2 seconds on 4G.
- Accessibility: All interactive elements must be keyboard-navigable. Markers need aria-labels.
- Browser support: Latest Chrome, Firefox, Safari, Edge. Mobile Safari and Chrome on iOS/Android.
- No backend required: All data is static and bundled at build time. No server, no database.
- Offline-friendly: Once loaded, the itinerary panel should work offline (map tiles require network).

10. Future Enhancements (Out of Scope for v1)

- Inline editing of trip data through the UI (drag-and-drop day reorder, edit forms).
- Weather forecast integration for each stop.
- Packing list and gear checklist per activity type.
- Cost tracker / budget breakdown.
- Photo upload per day after the trip.
- Export to Google Maps / Apple Maps for turn-by-turn navigation.
- Multi-trip support with route comparison.

Appendix A: Modifying the Schedule

When the trip plan changes, follow these steps:

8. Open src/data/tripData.ts.
9. If adding a new stop, first add a Location entry to the LOCATIONS object with the correct lat/lng.
10. Update or add the relevant TripDay entry in the days array.
11. Recalculate totalMiles and totalDrivingHours (or add a computed getter).
12. Save the file. The dev server will hot-reload and the map + itinerary will update immediately.

No component code needs to change. The map reads from TripContext which derives from the data file. This is by design.